

Putting together a multi-agent system

Time to combine everything we've learned into a robust, production-ready solution that juggles context, tools, and multiple agents: our own multi-agent system!

Our Deep Research multi-agent system will have three agents:

-  `QuestionAgent` which accepts a research topic and generates a bunch of questions
-  `AnswerAgent` which answers a specific question (we'll need to call it many times)
-  `ReportAgent` which aggregates all the answers and generates a report.

We'll create these as `FunctionAgent` s, the same as we did when creating agents for `AgentWorkflow` .

! Note: Run the hidden cells below before running the rest of the code. !

```
!pip install llama-index -q -q
```

```
!pip install tavily-python -q -q
```

```
from tavily import AsyncTavilyClient
from llama_index.core.agent.workflow import AgentWorkflow
from llama_index.llms.openai import OpenAI
import os
from openai import OpenAI as OpenAIClient

raw_client = OpenAIClient()

API_KEY = raw_client.api_key
API_BASE = raw_client.base_url

tavily_api_key = os.environ["TAVILY_API_KEY"]

async def search_web(query: str) -> str:
    """Useful for using the web to answer questions."""
    client = AsyncTavilyClient(api_key=tavily_api_key)
    return str(await client.search(query))

llm = OpenAI(model="gpt-4o-mini", api_key=API_KEY, api_base=API_BASE)
```

```

from llama_index.core.agent.workflow import FunctionAgent
from llama_index.llms.openai import OpenAI
import os

question_agent = FunctionAgent(
    tools=[],
    llm=llm,
    verbose=False,
    system_prompt="""You are part of a deep research system.
    Given a research topic, you should come up with a bunch of questions
    that a separate agent will answer in order to write a comprehensive
    report on that topic. To make it easy to answer the questions separately,
    you should provide the questions one per line. Don't include markdown
    or any preamble in your response, just a list of questions."""
)
answer_agent = FunctionAgent(
    tools=[search_web],
    llm=llm,
    verbose=False,
    system_prompt="""You are part of a deep research system.
    Given a specific question, your job is to come up with a deep answer
    to that question, which will be combined with other answers on the topic
    into a comprehensive report. You can search the web to get information
    on the topic, as many times as you need."""
)
report_agent = FunctionAgent(
    tools=[],
    llm=llm,
    verbose=False,
    system_prompt="""You are part of a deep research system.
    Given a set of answers to a set of questions, your job is to combine
    them all into a comprehensive report on the topic."""
)

```

The Workflow we'll need to handle this task needs to do a few things:

- Accept the topic and pass it to the QuestionAgent
- Take all the answers from the QuestionAgent and split them up, firing off one AnswerAgent for each question
- Aggregate all the questions and answers from the AnswerAgents
- Generate a single report from them

```

from llama_index.core.workflow import Event, Context
from llama_index.core.workflow import (
    StartEvent,
    StopEvent,
    Workflow,
    step
)

class GenerateEvent(Event):
    research_topic: str

class QuestionEvent(Event):
    question: str

class AnswerEvent(Event):
    question: str
    answer: str

class ProgressEvent(Event):
    msg: str

class DeepResearchWorkflow(Workflow):

    @step
    async def setup(self, ctx: Context, ev: StartEvent) -> GenerateEvent:
        self.question_agent = ev.question_agent
        self.answer_agent = ev.answer_agent
        self.report_agent = ev.report_agent

        ctx.write_event_to_stream(ProgressEvent(msg="Starting research"))

        return GenerateEvent(research_topic=ev.research_topic)

    @step
    async def generate_questions(self, ctx: Context, ev: GenerateEvent) ->
QuestionEvent:

        # CODE: set up context in the deep research workflow
        await ctx.set("research_topic", ev.research_topic)
        ctx.write_event_to_stream(ProgressEvent(msg=f"Research topic is
{ev.research_topic}"))

        # CODE: set the question agent
        result = await self.question_agent.run(user_msg=f"""Generate some questions
on the topic <topic>{ev.research_topic}</topic>.""")

        # Some basic string manipulation to get separate questions

```

```

lines = str(result).split("\n")
questions = [line.strip() for line in lines if line.strip() != ""]

# Record how many answers we're going to need to wait for
await ctx.set("total_questions", len(questions))

# Fire off multiple Answer Agents
for question in questions:
    ctx.send_event(QuestionEvent(question=question))

@step
async def answer_question(self, ctx: Context, ev: QuestionEvent) -> AnswerEvent:

    # CODE: set the answer agent
    result = await self.answer_agent.run(user_msg=f"""Research the answer to
this
    question: <question>{ev.question}</question>. You can use web
    search to help you find information on the topic, as many times
    as you need. Return just the answer without preamble or markdown.""")

    ctx.write_event_to_stream(ProgressEvent(msg=f"""Received question
{ev.question}
    Came up with answer: {str(result)}"""))

    return AnswerEvent(question=ev.question, answer=str(result))

@step
async def write_report(self, ctx: Context, ev: AnswerEvent) -> StopEvent:

    research = ctx.collect_events(ev, [AnswerEvent] * await
ctx.get("total_questions"))
    # If we haven't received all the answers yet, this will be None
    if research is None:
        ctx.write_event_to_stream(ProgressEvent(msg="Collecting answers..."))
        return None

    ctx.write_event_to_stream(ProgressEvent(msg="Generating report..."))

    # Aggregate the questions and answers
    all_answers = ""
    for q_and_a in research:
        all_answers += f"Question: {q_and_a.question}\nAnswer:
{q_and_a.answer}\n\n"

    # Prompt the report
    result = await self.report_agent.run(user_msg=f"""You are part of a deep
research system.
    You have been given a complex topic on which to write a report:
    <topic>{await ctx.get("research_topic")}</topic>.""")

```

Other agents have already come up with a list of questions about the topic and answers to those questions. Your job is to write a clear, thorough report that combines all the information from those answers.

Here are the questions and answers:

```
<questions_and_answers>{all_answers}</questions_and_answers>""")
```

```
return StopEvent(result=stop(result))
```

 **Note:** We've cached the report for the topic "History of San Francisco" to keep this solution fast so that you can see the outcome instantly.

If you'd like to run the full agentic workflow, you can run the code below. This will use the actual agents to generate the report from scratch.

We recommend this for your own projects when you're ready to explore the complete workflow, as it may involve more extensive processing times.

```
workflow = DeepResearchWorkflow(timeout=300)
handler = workflow.run(
    research_topic="History of San Francisco",
    question_agent=question_agent,
    answer_agent=answer_agent,
    report_agent=report_agent
)
```

 Copy

```
async for ev in handler.stream_events():
    if isinstance(ev, ProgressEvent):
        print(ev.msg)
```

```
final_result = await handler
print("==== The report ====")
print(final_result)
```

Let's take a look at how running it would get us a sequence of progress events plus our final report:

! Note: Run the hidden cell below before running the rest of the code. !

```
import json
import os
import asyncio

CACHE_PATH = "sf_history_cache.json"
CACHE_KEY = "History of San Francisco"

# Load or initialize the cache
if os.path.exists(CACHE_PATH):
    with open(CACHE_PATH, "r") as f:
        full_cache = json.load(f)
else:
    full_cache = {}

if CACHE_KEY in full_cache:
    cached_result = full_cache
```

```
async def replay_cached():
    messages = [
        "Starting research...",
        f"Research topic is: {CACHE_KEY}",
        "Collecting answers...",
        "Generating report..."
    ]
    for msg in messages:
        await asyncio.sleep(0.5)
        print(msg)

    print("==== The report ====")
    print(cached_result)

await replay_cached()
```

```
Starting research...
Research topic is: History of San Francisco
Collecting answers...
Generating report...
==== The report ====
# Comprehensive Report on the History of San Francisco
```

San Francisco, a city known for its iconic landmarks and vibrant culture, has a rich and complex history shaped by various events, movements, and demographic changes. This report explores the key historical milestones, social movements, economic transformations, and cultural developments that have defined San Francisco from its founding to the present day.

Founding and Early Development

The history of San Francisco begins long before European contact, with the Ohlone people inhabiting the region for thousands of years. The first significant European interest in the area began with Spanish exploration, notably in 1775 when the Spanish ship San Carlos entered San Francisco Bay. This led to the establishment of the Presidio of San Francisco and Mission San Francisco de Asís in 1776, marking the beginning of Spanish colonial presence.

Following Mexico's independence from Spain in 1821, the area remained relatively undeveloped until the California Gold Rush in 1848. The discovery of gold transformed San Francisco from a modest settlement of about 200 residents into a bustling metropolis, with the population skyrocketing to approximately 25,000 by 1849. This rapid growth established San Francisco as a major commercial and financial center on the West Coast, officially incorporating as a city in 1850.

